

ФАЛЬКОН-ReID — сопроводительная документация

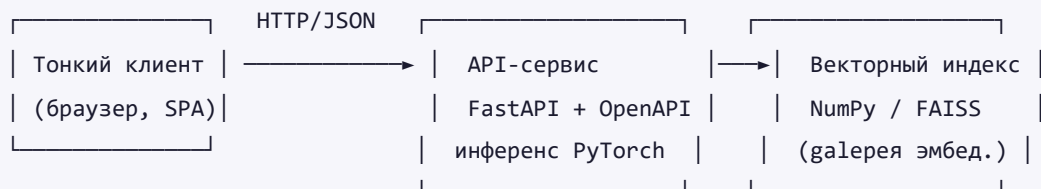
Сервис формирования цифрового признака транспортного средства для сопоставления снимков одного автомобиля из разных локаций без использования государственного номера. Кейс «Фалькон Тех», ЛЦТ-2026.

1. Функциональная архитектура

Сервис реализует четыре этапа, требуемые ТЗ:

- 1. Этап получения.** REST API (`POST /api/search` , `POST /api/embed`) и веб-интерфейс принимают изображение (полный кадр или кроп) и координаты BBox `x, y, w, h` . Валидация: декодируемость изображения, положительные размеры рамки, попадание рамки в границы кадра. Некорректный вход → HTTP 400 с объяснением.
- 2. Этап обработки.** Кроп по BBox с запасом 6% (контекст: колёса, тень, стойки кузова), нормализация ImageNet. Четверной ансамбль — DINOv2 ViT-L/14 (336^2), DINOv2 ViT-B/14 с части-признаками (252^2), ConvNeXt-base (224^2) и DINOv3 ViT-L/16 (256^2), каждая модель с BNNeck и TTA-отражением — даёт векторы $768 + 1536 + 768 + 768$; взвешенная конкатенация (1.0/0.7/0.5/1.0) и L2-нормировка формируют итоговый эмбединг **3840-d float32**. Постобработка (подобрана на валидации): DBA $k=3$ расширяет векторы галереи соседями по галерее, alphaQE $k=1$ расширяет запрос топ-1 галереи; итог снова L2-нормируется.
- 3. Этап анализа.** Косинусная близость запроса ко всем векторам галереи (скалярное произведение нормированных векторов). Для галерей $\sim 10^6$ предусмотрен приближённый поиск FAISS HNSW (см. §7).
- 4. Этап результата.** Сортировка по убыванию близости; выдача топ-N с численной уверенностью. Если максимальная близость ниже порога τ — **пустой ответ (отказ)** с человекочитаемым сообщением.

2. Компонентная архитектура (микросервисы)



- **Backend инференса** — Python 3.11+, PyTorch, FastAPI; формализованное OpenAPI-описание доступно на `/docs` (Swagger UI) и `/openapi.json` .

- **Хранилище векторов** — FAISS `IndexFlatIP` (точный косинусный поиск, специализированная векторная БД по T3); векторы и метаданные галереи — `gallery_index.npz`. На городском масштабе индекс заменяется FAISS HNSW (демонстрация: `src/ann_demo.py`); схема расширяется на PostgreSQL+pgvector без изменения API.
- **Клиентская часть** — статический SPA (HTML/CSS/JS, без сборки), отдаётся тем же сервисом; работает в любом современном браузере.
- Все компоненты упакованы в Docker; запуск — `docker compose up`, офлайн (веса в составе образа).

3. Методология и модель

3.1 Выбор backbone

Четверной ансамбль разнородных архитектур (манифест `models/ensemble.json`):

- `vit_large_patch14_reg4_dinov2.lvd142m` (304M, вес 1.0, вход 336^2) — самообученные признаки DINOv2, сильная инициализация для fine-grained retrieval, что критично при 9.5k обучающих снимков. Обучена с gradient checkpointing (12 ГБ GPU). Вход 336^2 вместо 224^2 даёт модели микродетали (наклейки, повреждения, диски): соло 0.772 → 0.795 mAP.
- `vit_large_patch16_dinov3.lvd1689m` (305M, вес 1.0, вход 256^2) — DINOv3, следующее поколение самообучения Meta (2025): другой корпус (LVD-1689M) и метод — ошибки декоррелированы с DINOv2. В одиночку: mAP 0.785 — сильнейшая одиночная модель; в ансамбле даёт +2.8 mAP.
- `vit_base_patch14_reg4_dinov2.lvd142m` + **части-признаки** (87M, вес 0.7, вход 252^2) — patch-токены делятся на 3 горизонтальные полосы (перед/центр/ зад кузова), у каждой свой 256-d эмбединг с BNNeck и CosFace-головой (`ReIDModelParts`). Против «близнецов» — одинаковых моделей одного цвета: соло 0.751 → 0.777 mAP.
- `convnext_base.fb_in22k_ft_in1k` (89M, вес 0.5) — свёрточная сеть с другим индуктивным смещением.
- Веса хранятся в **fp16** (инференс и так идёт в autocast-fp16; $\cos(\text{fp16}, \text{fp32}) \geq 0.99998$ на реальных кропах — `src/fp16_convert.py`): файлы **1505 МБ** при лимите 2048 МБ. Веса конкатенации выбраны перебором сеток (`runs/triple_eval.json`, `runs/quad_eval.json`).

Абляции (валидация, кросс-камерный mAP, всё с DBA/AQE): ViT-L 0.772 → DINOv3-L 0.785 → тройной L+0.7B+0.7C 0.807 → четверной L+0.7B+0.5C+DINOv3 0.836 → ViT-L@336 + ViT-B с части-признаками **0.844**. Таблицы: `runs/quad_eval.json`, `runs/triple_eval.json`, `runs/ensemble_eval.json`.

3.2 Обучение

- **Функция потерь**: CE по 1541 классу с отступом CosFace ($s=48$, $m=0.25$) + label smoothing 0.1, плюс batch-hard triplet ($m=0.3$) на признаках до BNNeck. Отступ формирует компактные кластеры идентичностей → лучшее разделение «свой/чужой» в open-set.
- **BNNeck** (Luo et al., 2019) разводит метрическое пространство triplet и классификационное; на инференсе используется выход BN.

- **Сэмплирование РК 16×4:** каждый батч — 16 идентичностей × 4 снимка, гарантирует позитивы/негативы для triplet.
- **Аугментации:** горизонтальное отражение, color jitter (яркость/контраст/насыщенность), gaussian blur (имитация расфокуса камер), random erasing (имитация перекрытий).
- **Оптимизация:** AdamW (lr 1e-4 голова, 1e-5 backbone, wd 0.05), косинусный график с 2 эпохами прогрева, 30 эпох, AMP fp16.
- Сиды зафиксированы; разбиение и результат воспроизводимы.

3.3 Инференс и постобработка

Эмбединги после BNNeck + TTA-flip каждой модели, взвешенная конкатенация (1.0 / 0.7 / 0.5 / 1.0), L2-нормировка. Затем классические методы поиска: **DBA k=3** (database augmentation: вектор галереи заменяется взвешенной суммой с ближайшими соседями по галерее, alpha=3) и **alphaQE k=1** (запрос расширяется топ-1 галереей). Подбор по сетке на валидации: +3.4 mAP при неизменном качестве режима отказа; плато широкое (0.74–0.75 для любых k), выбор не хрупкий. Итоговые векторы записываются в `embeddings.npy`; `submission.csv` — это в точности косинусное ранжирование сохранённых векторов. Батч=1 латентность и пакетный FPS — `artifacts/benchmark.json`.

4. Валидационный протокол

Разбиение по идентичностям 80/20 (1233 train / 308 val), val-ТС модель никогда не видела — как в тесте организаторов. Из val собирается протокол:

- для каждой идентичности камеры делятся между галереей и запросами (кросс-камерное сопоставление);
- **35% идентичностей не попадают в галерею вовсе** — их запросы обязаны получать отказ (open-set негативы, аналог скрытого теста, где запросов в 1.5 раза больше, чем объектов галереи);
- совпадения «та же камера» исключаются из метрик.

Метрики: mAP, Rank-1, Rank-5, mINP — `artifacts/val_report.json`.

5. Режим предложения кандидатов и обоснование порога

Уверенность = косинусная близость (монотонна, интерпретируема, стабильна между запусками).

Одно-сценные дубли. Тестовая галерея содержит кадры тех же сцен, что и запросы: 72% запросов имеют на топ-1 «совпадение» с той же камерой (соседний по времени кадр или та же припаркованная машина). Суть ReID — кросс-камерный поиск, поэтому такие кандидаты исключаются. Камеры-группы восстанавливаются без разметки: (а) фоновые дескрипторы кадров (зона ТС маскируется), связные компоненты при $\cos \geq 0.8$ — точность на train-разметке 1.0; (б) «та же

парковка»: фон ≥ 0.5 и IoU BBox ≥ 0.85 (или фон ≥ 0.3 и IoU ≥ 0.9) — точность ≥ 0.97 . На валидации правило не меняет F1/TNR (ложных исключений настоящих кросс-камерных пар нет).

Кандидаты с $\text{cos} \geq \tau$ среди кросс-сценных принимаются; если таких нет — отказ.

Выбор τ : по сетке из 501 порога на валидации считаются F1 (по паре запрос-кандидат топ-1) и TNR (доля корректных отказов на дистракторах). τ = максимум F1 **при ограничении $\text{TNR} \geq 0.70$:** безусловный максимум F1 достигается при $\text{TNR} \approx 0.3$ (70% ложных принятий на дистракторах) — сервис безопасности не должен подсовывать оператору ложные «находки», цена ложного принятия выше цены пропуска. Выбранная точка (четверной ансамбль + DBA/AQE, валидация с инференс-камерами — как на тесте): **$\tau=0.320$, $\text{F1}=0.768$, $\text{TNR}=0.739$** ; с истинными камерами — F1 0.781, TNR 0.772. Числа: `models/threshold.json`, кривые: `models/val_curves.json`. Альтернативные меры уверенности (margin, z-score, цветовое veto) проверены и отклонены — `runs/confidence_tune.json`.

Порог — параметр развёртывания (`REFUSAL_THRESHOLD`), оператор может менять его в UI на лету; значение по умолчанию — обоснованный τ .

6. Анализ ошибок

Автоматический разбор — `src/eval_holdout.py` → `artifacts/error_cases.json`:

- **rank1_miss** — запросы, где верный кандидат не на первом месте (типично: одинаковые массовые модели одного цвета, разворот 180°);
- **false_accept_distractor** — дистракторы, преодолевшие порог (близнецы по модели/цвету/комплектации);
- **hard_correct** — верные совпадения с низкой уверенностью (ночь, дальний ракурс, перекрытия).

Выводы и примеры — в презентации; направления развития: локальные части-признаки (повреждения, наклейки), метрическое дообучение на hard-негативах, мультикадровая агрегация треков.

7. Масштабирование

- Галерея 750 — точный поиск за миллисекунды.
- Городской масштаб 10^6 : FAISS HNSW32 + int8-квантизация (SQ8) — 0.4 мс/запрос при $\text{recall}@10$ 0.88 (0.66 мс → 0.91), ускорение $\times 23$ к точному поиску; индекс миллиона 1536-d векторов занимает **1.4 ГБ RAM** (для 3840-d ансамбля — пропорционально ~ 3.6 ГБ, либо PCA до 512-d). Замер: `artifacts/ann_demo.json`; галерея демо построена на геометрии реальных эмбедингов теста.
- Горизонтально: stateless API-реплики за балансировщиком; индекс — в выделенном сервисе (Milvus/pgvector) с шардированием по времени/округам.

- Пакетная обработка потока: динамическое батчирование инференса (очередь), замер FPS — `src/benchmark.py`.

8. Запуск и воспроизведение

См. README.md (быстрый старт, обучение, все команды). Кратко:

```
DATA_DIR=/данные docker compose up --build      # сервис+UI: localhost:8000
DATA_DIR=/данные docker compose run --rm inference # артефакты сдачи
```

9. Соответствие требованиям и ограничениям

Требование ТЗ	Выполнение
Эмбеddинг фиксированной размерности	3840-d float32, L2-норм
Мера близости	косинус
Топ-N + отказ	<code>/api/search</code> , порог τ обоснован
OpenAPI	<code>/docs</code> , <code>/openapi.json</code>
Тонкий клиент	SPA в браузере
Docker, одна команда	<code>docker compose up</code>
Офлайн-инференс	веса в образе, <code>HF_HUB_OFFLINE=1</code>
Веса ≤ 2 ГБ	1505 МБ (четверной ансамбль, fp16)
Без признаков номера	номера размыты в данных; attention-карты подтверждают
Python/открытое ПО	PyTorch, FastAPI, FAISS (BSD/MIT/Apache)

10. Список внешних библиотек и датасетов

Полный список с версиями — `requirements.txt` и README.md §«Внешние ресурсы». Сторонние датасеты не использовались; предобученные веса — общедоступные DINOv2 (Apache-2.0).